

ZAFEER TARIQ

+92-300-4220223 ◇ Lahore

[E-mail](#) ◇ [Portfolio](#) ◇ [LinkedIn](#) ◇ [GitHub](#)

EDUCATION

Bachelors of Science in Computer Science, FAST-NUCES, Lahore
3.58 / 4.00

2021–2025

EXPERIENCE

Associate Game Developer

July 2025–Present

Tapfire Games, Lahore

- Developed single-player and multiplayer idle games in Unity for HTML5/web platforms, optimizing builds for both mobile and desktop browsers while maintaining lightweight game sizes and smooth performance.
- Built multiplayer games in Cocos Creator with integration to custom backend servers and ensuring efficient data handling for real-time gameplay.
- Designed and implemented shaders in both Unity and Cocos to enhance visual performance and effects.

Teaching Assistant

Aug 2024–June 2025

FAST-NUCES, Lahore

- Teaching Assistant for Data Structures during the Fall 2024 semester. Helped the student with programming assignments about complex data structures.
- Teaching Assistant for Compiler Construction for the Spring 2025 semester.

Intern Game Developer

Summer 2024

TheHexaTown, Lahore

- Worked on Indie-style games in Unity.
- Developed Unity shaders in ShaderLab for various visual effects.

SKILLS

Programming Languages
Game Development
Technologies

C/C++, Python, C#, Typescript, Lua, SQL
Unity Game Engine, Cocos Creator, HLSL, ShaderLab, SFML, LOVE2D
CUDA, Linux, Firebase, Git/Github

HONOURS AND AWARDS

- **Dean's List Recognition (x5)**. Inscribed in the Dean's List of Honour for achieving a GPA of 3.5 or higher during the semester.

PROJECTS

My Perfect Theme Park

- Implemented core gameplay systems in Unity (C#) including visitor flow, ride logic, and queue-based interactions.
- Developed resource collection and interaction mechanics with animations, VFX/SFX feedback, and state-based object behavior.
- Built progression systems covering area unlocking, upgrades, and XP-driven feature scaling.
- Programmed AI-driven visitor behavior including dynamic ride selection, capacity handling, and automated income generation.

- Integrated idle/automation systems such as helpers for rides, ticketing, and resource collection.

Archery King

- Multiplayer idle archery game (HTML5/Unity) with turn-based mechanics.
- Implemented tight aiming and shooting systems for precision gameplay.
- Implemented smooth camera transitions and movement systems to enhance gameplay experience.

Turtle Fight

- Competitive lane-based game featuring class-based turtles with unique stats.
- Implemented an adaptive AI system that considers player level, turtle upgrades, and real-time lane strength to counter player strategies.
- Focused heavily on game feel (“juice”), including screen shake, collision-driven VFX, smooth Spine animations, multiple skins, and themed maps.

Carrom Stars

- Classic carrom game with a league-based progression system, in-game currency, and cosmetic unlocks.
- Made a geometry-based AI opponent using the law of cosines to evaluate striker–puck–pocket paths and select optimal shots.
- Designed core gameplay systems including entry fees, rewards, and AI difficulty scaling to support competitive play.

Shader Development

- Written shaders for different lighting techniques like Lambert and Phong/Blinn-Phong Reflections.
- Various shaders for visual effects like Fresnel Highlight and Transparency Effects.
- Limited experience with Compute and Surface Shaders as well.

Personal Unity Projects

- Isle Of Shadows - Survival game with procedurally generated islands. ([GitHub](#))
- Different games in Unity including an Endless Runner Game with procedural obstacle spawning, a simple FPS prototype and a 2D Platformer.

Bezier Curve Editor ([GitHub](#))

- A simple bezier curve editor with varying number of control points and immediate curve rendering.

FINAL YEAR PROJECT

Design and Development Of a Quantum Compiler To Maximize Parallelism.

- Wrote a compiler to parse a subset of the OpenQASM language and generate a Parse Tree.
- Simulated the Quantum Gates in C++ using NVIDIA CUDA for GPU based parallelism.
- Designed algorithms to expand simple gates based on the state vector size and control gates based on the state vector size and the control and target qubits.
- Used strategies to limit the CPU-GPU transfer overhead.
- Implemented many famous Quantum Algorithms like the Grover’s Search Algorithm, Quantum Fourier Transformation, Quantum Phase Estimation etc. to benchmark the performance.